CSE335s: Operating Systems (Spring26)

CPU Scheduler Project

Team members

Name	ID
Amr Ashraf Hussien	2201048
Youssef Ahmed Mohammed	2200405
Mohammed Hamada Hassan	2200820
Mohammed Walid Abd Elmohsen	2200243
Ahmed Mohamed Mohyeldin	2200955
Ahmed Mahmoud Elmorsy	2200725

Delivering to: Dr/ **Sahar M. Mahmoud Haggag**

Introduction

CPU scheduling is a fundamental concept in operating systems that is responsible for managing the execution of processes by the central processing unit (CPU). Since multiple processes may be ready to run at the same time, the operating system must decide which process should be executed first and for how long. This decision is made using scheduling algorithms.

The main goal of CPU scheduling is to optimize system performance by minimizing waiting time, reducing turnaround time, and maximizing CPU utilization. Different scheduling algorithms follow different strategies to achieve these goals, and each algorithm has its own advantages and disadvantages depending on the scenario.

In this project, several CPU scheduling algorithms are implemented, including First Come First Serve (FCFS), Shortest Job First (SJF) in both preemptive and non-preemptive forms, Priority Scheduling (preemptive and non-preemptive), and Round Robin. The project provides a graphical user interface (GUI) that allows users to input processes, simulate scheduling in real time, and visualize the execution using a Gantt chart.

Additionally, the system calculates important performance metrics such as average waiting time and average turnaround time, helping to compare the efficiency of different scheduling techniques. This project aims to provide a clear understanding of how CPU scheduling works and how different algorithms impact system performance.

Algorithms Implemented

In this project, several CPU scheduling algorithms are implemented to simulate and analyze their behavior under different conditions. Each algorithm follows a different strategy to decide the execution order of processes.

1. First Come First Serve (FCFS)

FCFS is the simplest scheduling algorithm. Processes are executed in the order they arrive in the ready queue. Once a process starts execution, it runs until completion without interruption.

Advantages:

- **Simple and easy to implement**
- **No starvation (every process gets executed)**

Disadvantages:

- **Poor performance in terms of waiting time**
- **Causes the "convoy effect" when long processes delay shorter ones**

2. Shortest Job First (SJF)

a) Non-Preemptive SJF

In this version, the process with the smallest burst time is selected for execution. Once it starts, it runs until it finishes.

b) Preemptive SJF

In this version, the currently running process can be interrupted if a new process arrives with a shorter remaining burst time.

Advantages:

- **Minimizes average waiting time**
- **Efficient for batch systems**

Disadvantages:

- **Requires knowing burst time in advance**
- **May cause starvation for longer processes**

3. Priority Scheduling

a) Non-Preemptive Priority

Processes are executed based on priority. The process with the highest priority (lowest number) is selected first and runs until completion.

b) Preemptive Priority

A running process can be interrupted if a new process arrives with a higher priority.

Advantages:

- **Important processes are executed first**
- **Flexible scheduling**

Disadvantages:

- **Starvation may occur for low-priority processes**
- **Requires priority management**

4. Round Robin (RR)

Round Robin is a time-sharing scheduling algorithm where each process is assigned a fixed time slice (quantum). Processes are executed in a circular order. If a process does not finish within its time quantum, it is moved to the end of the queue.

Advantages:

- **Fair allocation of CPU**
- **Suitable for time-sharing systems**

Disadvantages:

- **Performance depends on the chosen quantum**
- **Too small quantum increases overhead, too large makes it similar to FCFS**

System Design

The system is implemented as a desktop application using C++ and the Qt framework. It simulates CPU scheduling algorithms through an interactive graphical user interface and supports both static and live scheduling modes.

Architecture Overview:

The system follows a modular architecture consisting of three main layers:

1. User Interface Layer (GUI)

This layer is implemented using Qt Widgets and is managed by the MainWindow class. It handles all user interactions such as:

- Selecting the scheduling algorithm using a ComboBox
- Entering process data (Process ID, Arrival Time, Burst Time, Priority)
- Displaying processes in a table
- Showing the Gantt Chart dynamically
- Displaying results such as average waiting time and turnaround time

2. Scheduling Layer (Algorithms)

This layer contains the implementation of different scheduling algorithms. Each algorithm is implemented as a separate class, including:

- FCFS
- SJF(Non-preemptive)
- SJF (Preemptive SJF)
- Priority_(Preemptive Priority)
- Priority_ (Non-preemptive Priority)
- RoundRobin

All these classes follow a common interface (via Scheduler.hpp) and implement core functions such as:

- Schdule() → returns the currently running process ID
- isFinished() → checks if all processes are completed
- Average_wait_time()
- Average_turnaround_time()

3. Data Management Layer

This layer manages process data using:

- A Process class (contains ID, arrival time, burst time, priority)
- A processlist vector to store all processes
- A sorter component that:
    - Sorts processes based on arrival time
    - Transfers ready processes to the scheduler dynamically using
    - transferToScheduler()

Live Scheduling Mechanism:

The system simulates real-time scheduling using a QTimer:

- Each time unit = 1 second
- The function ontick() is triggered every second
- At each tick:
    - New processes are transferred to the scheduler
    - The scheduler selects the next process
    - Remaining burst time is updated in the table
    - The Gantt chart is updated dynamically

Gantt Chart Visualization:

The Gantt Chart is implemented using a QScrollArea with dynamically created QLabel blocks:

- Each process is assigned a unique color
- Each block represents execution over time
- Blocks expand dynamically based on execution duration
- Idle time is also visualized

Modes of Operation:

1. Live Scheduling

- Runs in real-time using QTimer
- Allows dynamic addition of processes during execution
- Updates UI continuously

2. Static Scheduling

- Runs the full simulation instantly using a loop
- No real-time delay
- Displays final results directly
- User Interaction Flow
- User selects scheduling algorithm
- User enters process data and clicks "Add Process"
- Processes are stored and displayed in the table
- User chooses:
- Live Scheduling → simulation runs in real time
- Static Scheduling → simulation runs instantly
- Results and Gantt chart are displayed

Key Features in Implementation:

- Dynamic enabling/disabling of input fields based on algorithm
- Real-time simulation using QTimer
- Dynamic process insertion during execution
- Automatic update of arrival time during live scheduling
- Pause and Resume functionality

- Reset functionality to restart simulation

Technologies Used

- Programming Language: C++
- Framework: Qt (Qt Widgets)
- Core Components:
  - QMainWindow
  - QTimer
  - QTableWidget
  - QScrollArea

System Features

The implemented system provides a set of advanced features that enhance usability and accurately simulate CPU scheduling behavior.

1. Multiple Scheduling Algorithms

The system supports multiple CPU scheduling algorithms, including:

- First Come First Serve (FCFS)
- Shortest Job First (Preemptive and Non-Preemptive)
- Priority Scheduling (Preemptive and Non-Preemptive)
- Round Robin

This allows users to compare different scheduling techniques within the same environment.

2. Real-Time (Live) Scheduling

The system simulates scheduling in real time using a timer mechanism where:

- Each time unit corresponds to one second
- The system updates continuously during execution
- Processes are executed step-by-step as in real operating systems

3. Static Scheduling Mode

In addition to live simulation, the system provides a static mode where:

- All processes are executed instantly
- Results are displayed without delay
- Useful for quick analysis and testing

4. Dynamic Process Insertion

Users can add new processes while the scheduler is running:

- Newly added processes are automatically inserted into the system
- Arrival time is handled dynamically based on the current simulation time

5. Interactive Graphical User Interface (GUI)

The system includes a modern and user-friendly GUI that allows:

- Easy input of process data
- Selection of scheduling algorithms
- Real-time visualization of results

6. Gantt Chart Visualization

The execution of processes is visualized using a dynamic Gantt chart:

- Each process is represented with a unique color
- Execution intervals are displayed clearly
- The chart updates in real time during simulation

7. Remaining Burst Time Tracking

The system continuously updates the remaining burst time for each process:

- Displayed in a table
- Updated at every time step
- Helps users track execution progress

8. Performance Metrics Calculation

The system calculates important scheduling metrics, including:

- Average Waiting Time
- Average Turnaround Time

These metrics are displayed automatically after execution finishes.

9. Pause and Resume Functionality

The user can pause and resume the simulation at any time:

- Allows better observation of scheduling behavior
- Maintains system state during pause

10. Intelligent Input Handling

The system enables or disables input fields dynamically based on the selected algorithm:

- Quantum time is only enabled for Round Robin
- Priority input is only enabled for Priority scheduling
- Prevents invalid or unnecessary input

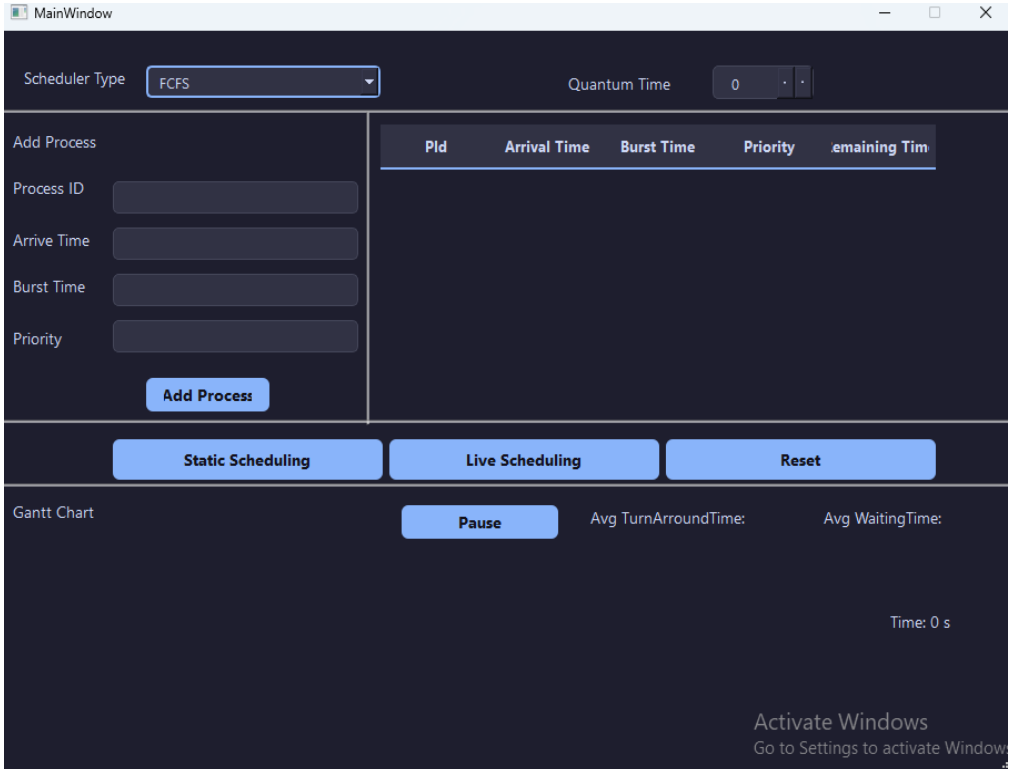
11. Reset Functionality

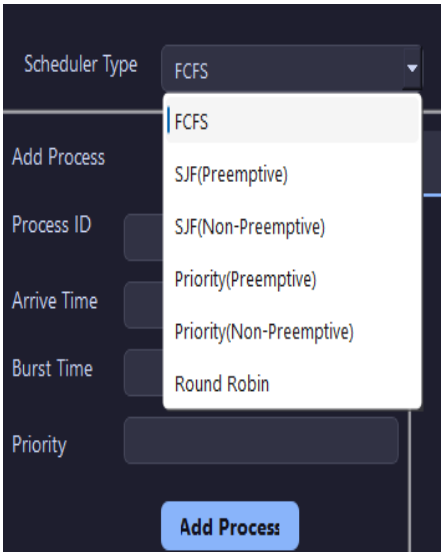
The system provides a reset option that:

- Clears all processes
- Resets the timer
- Clears the Gantt chart
- Restores the system to its initial state

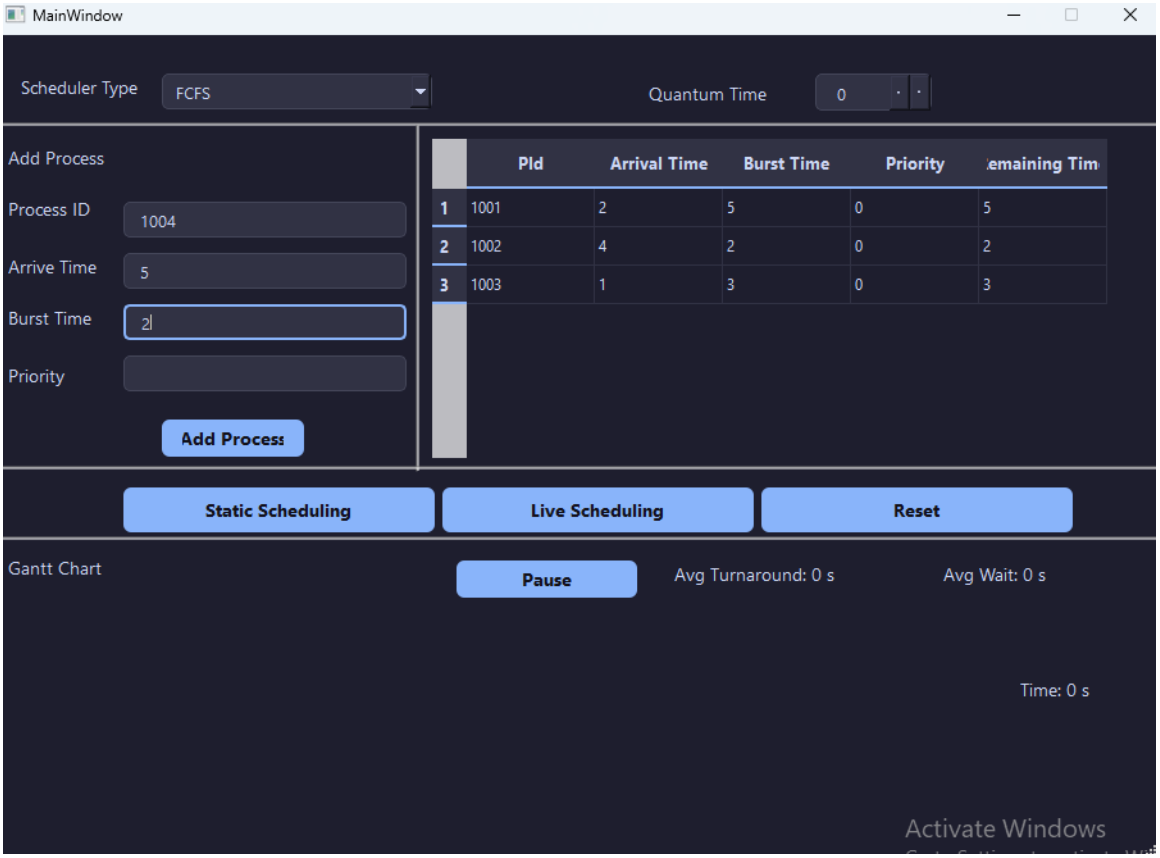
Screenshots

1) Main Interface (Before Adding Processes)



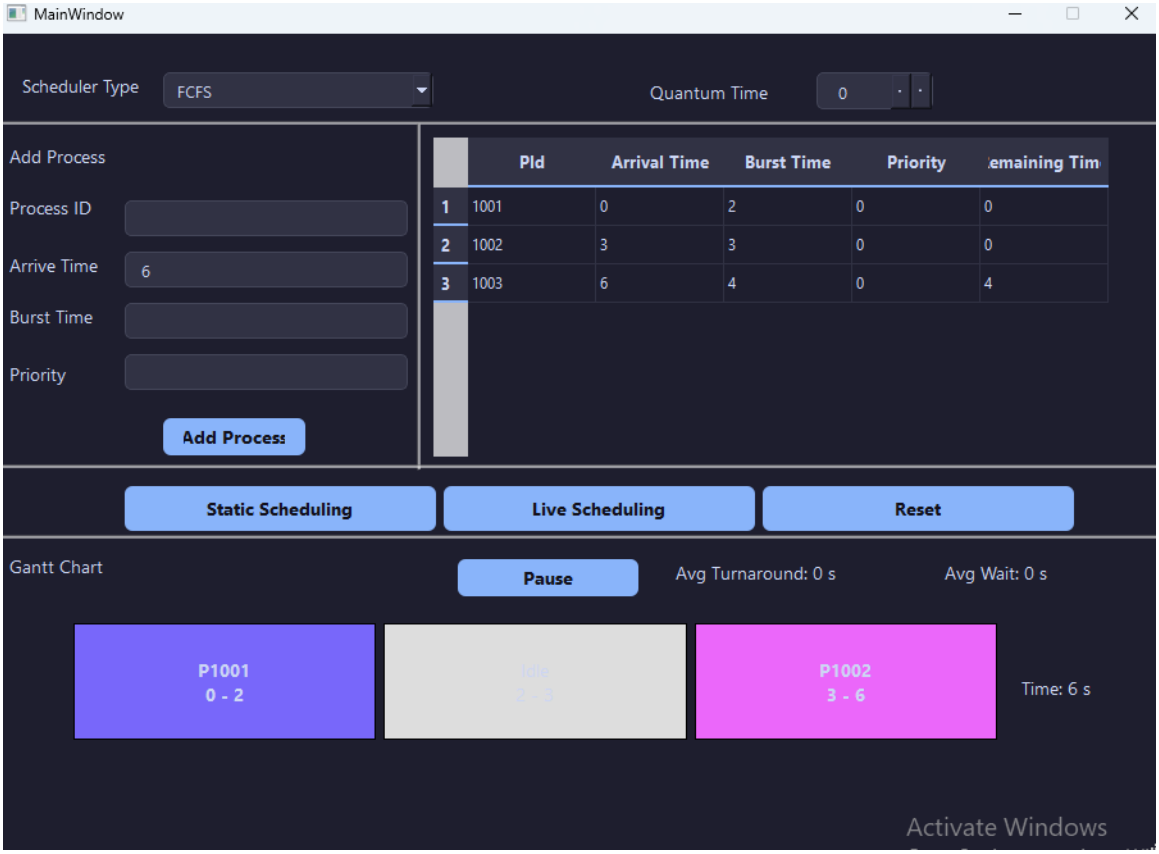


2) Adding processes



Processes added to the system and displayed in the table

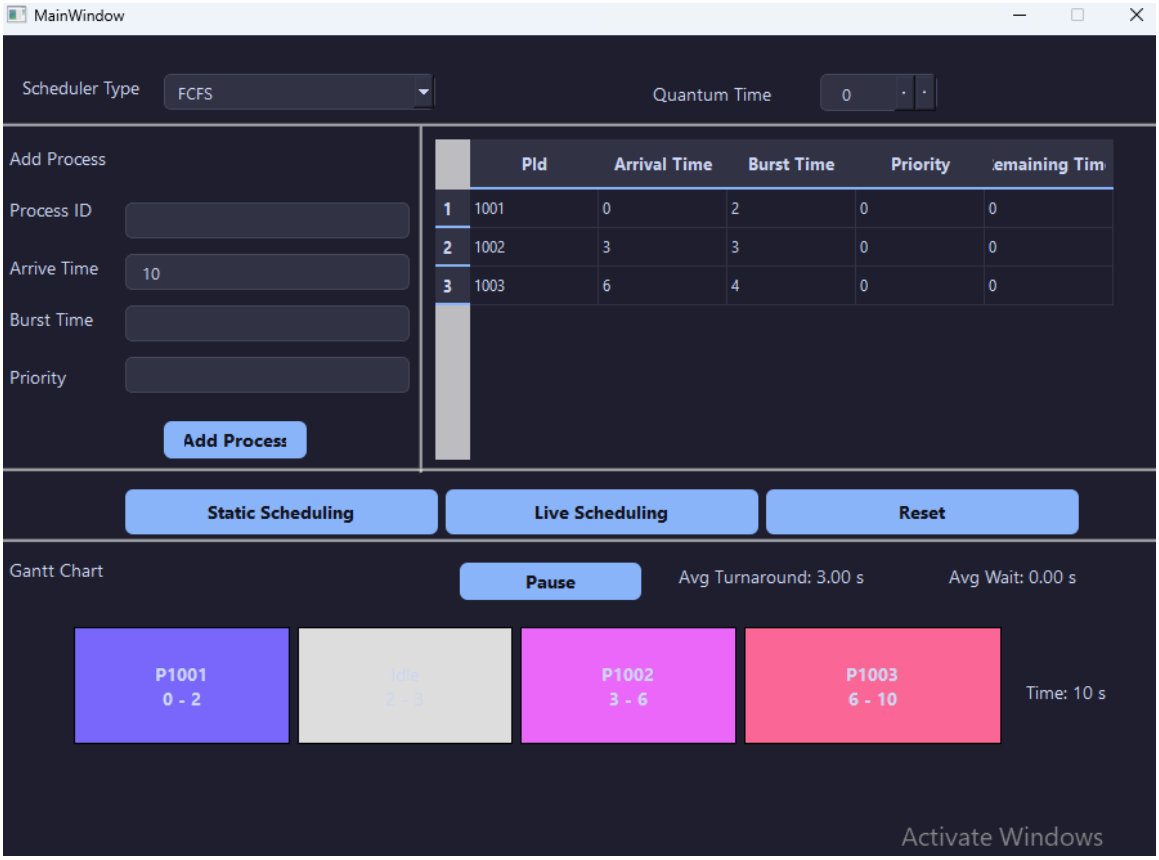
3) Live Scheduling



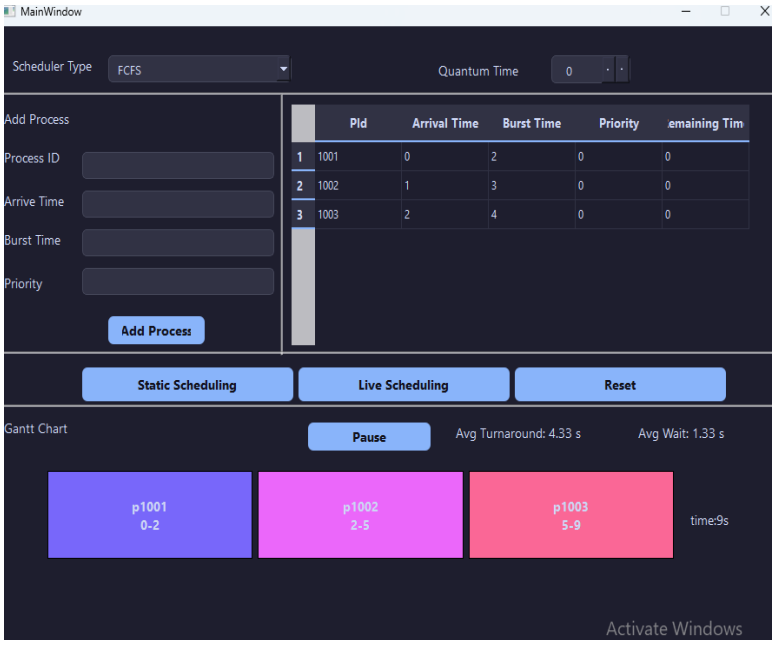
Live scheduling simulation with real-time updates

Dynamic Gantt chart representing process execution order

4)Final result

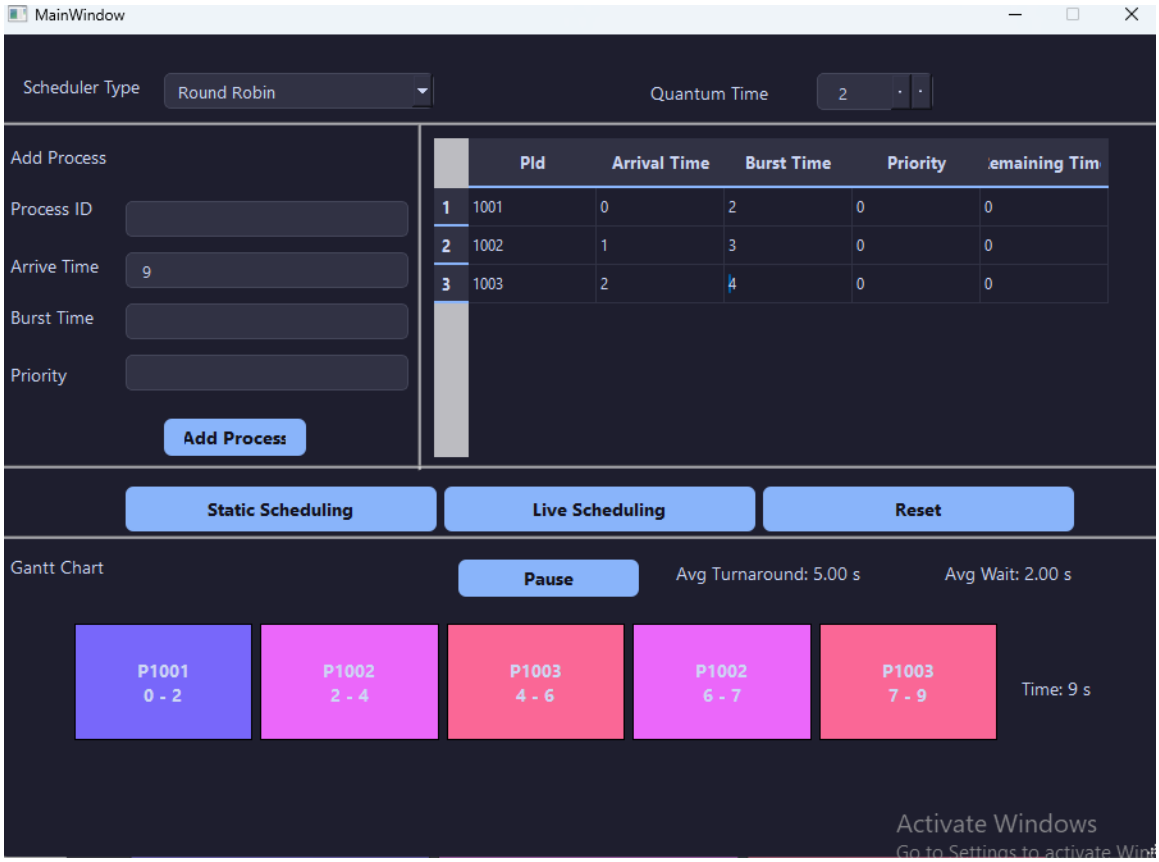


Final performance metrics after simulation



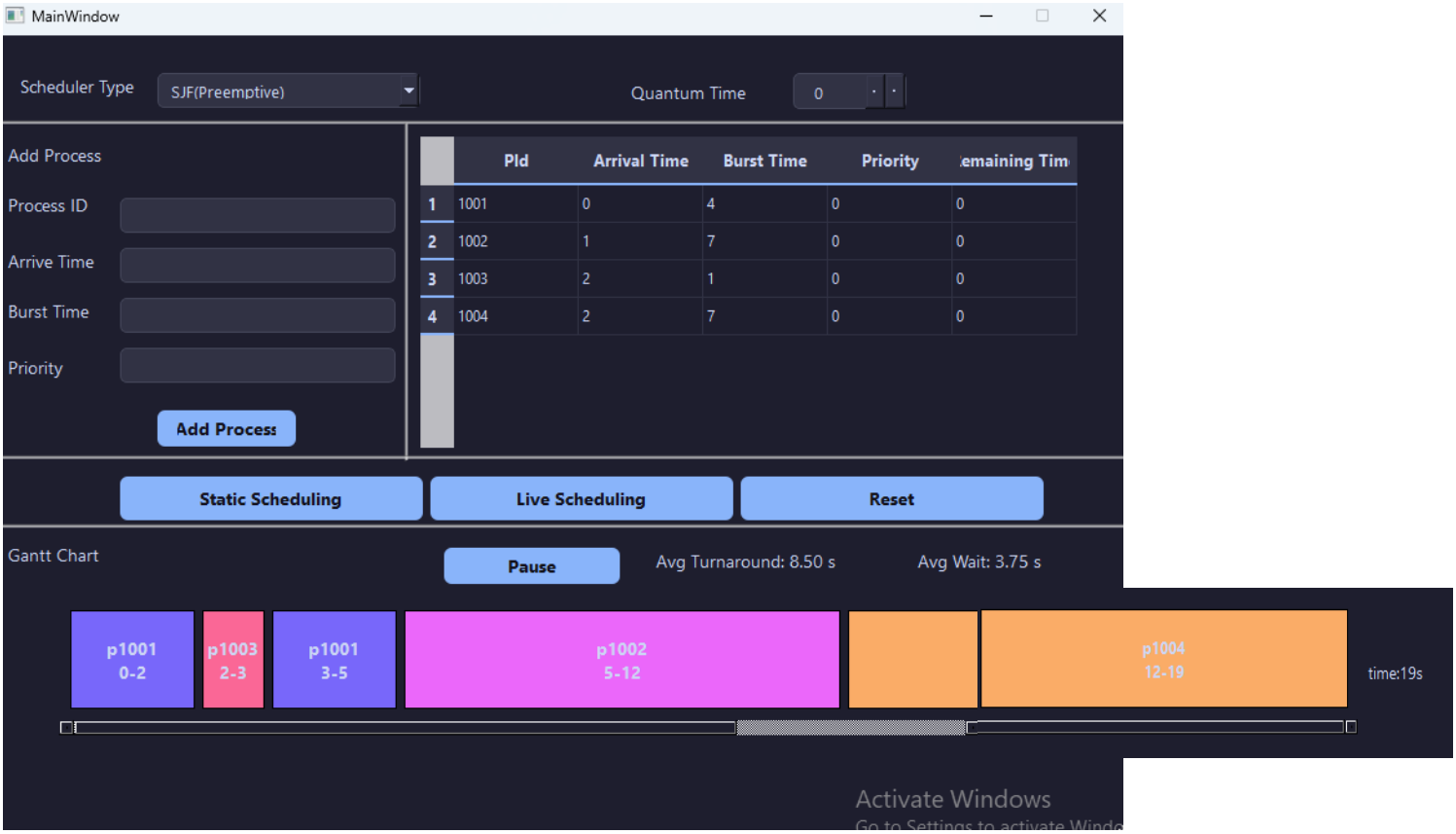
Another example have an avg waiting time

5) Round Robin

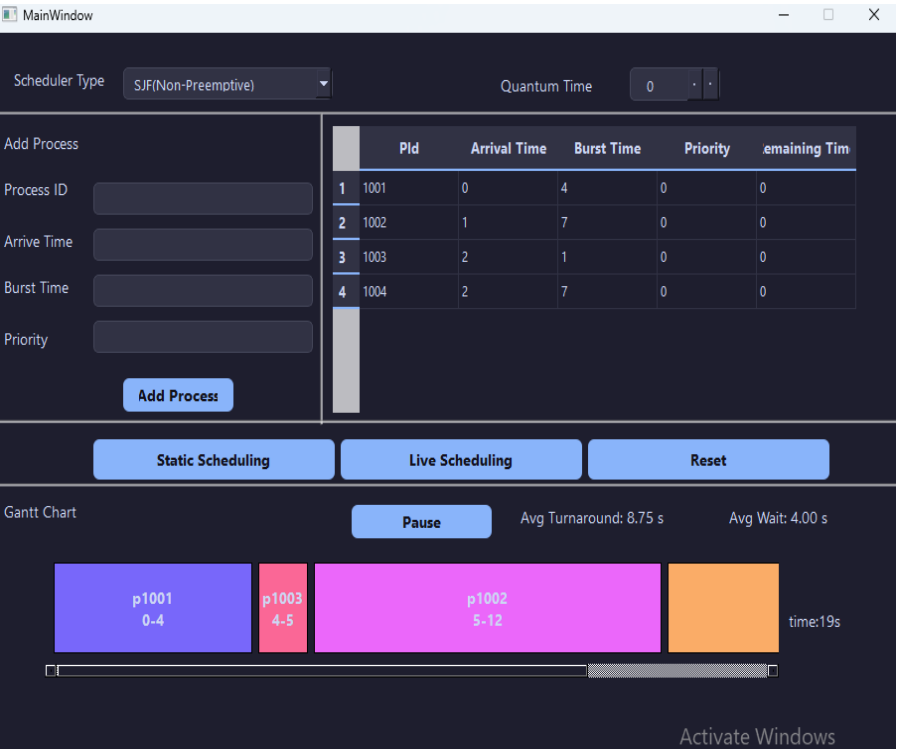


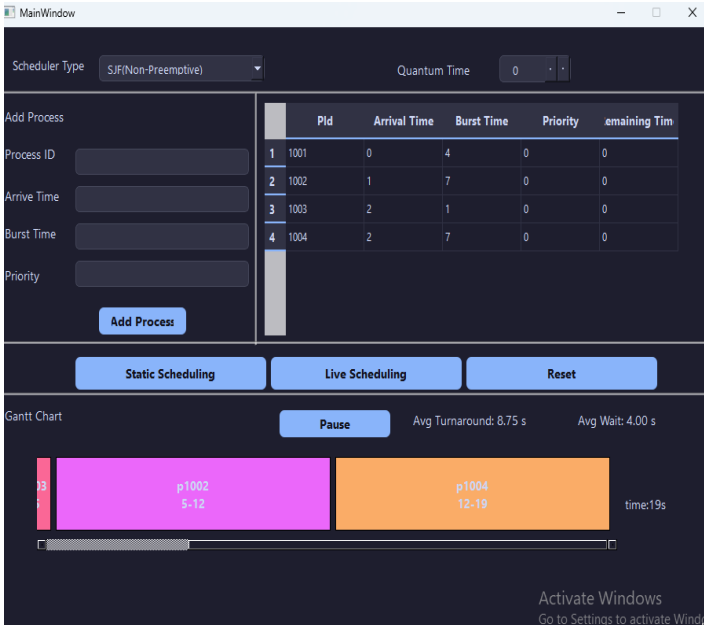
Round Robin scheduling with time quantum

6) SFJ preemptive

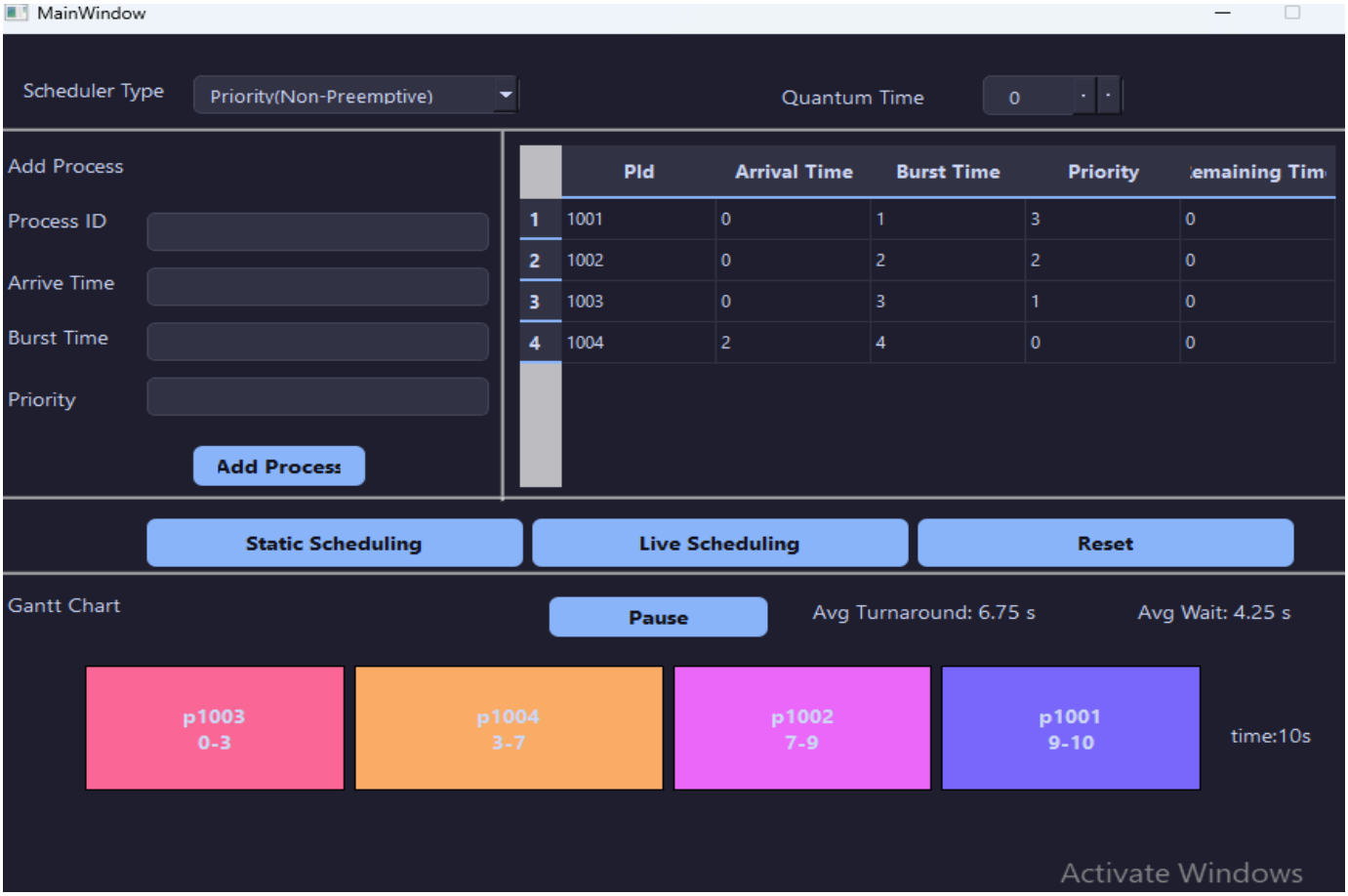


7) SFJ non preemptive

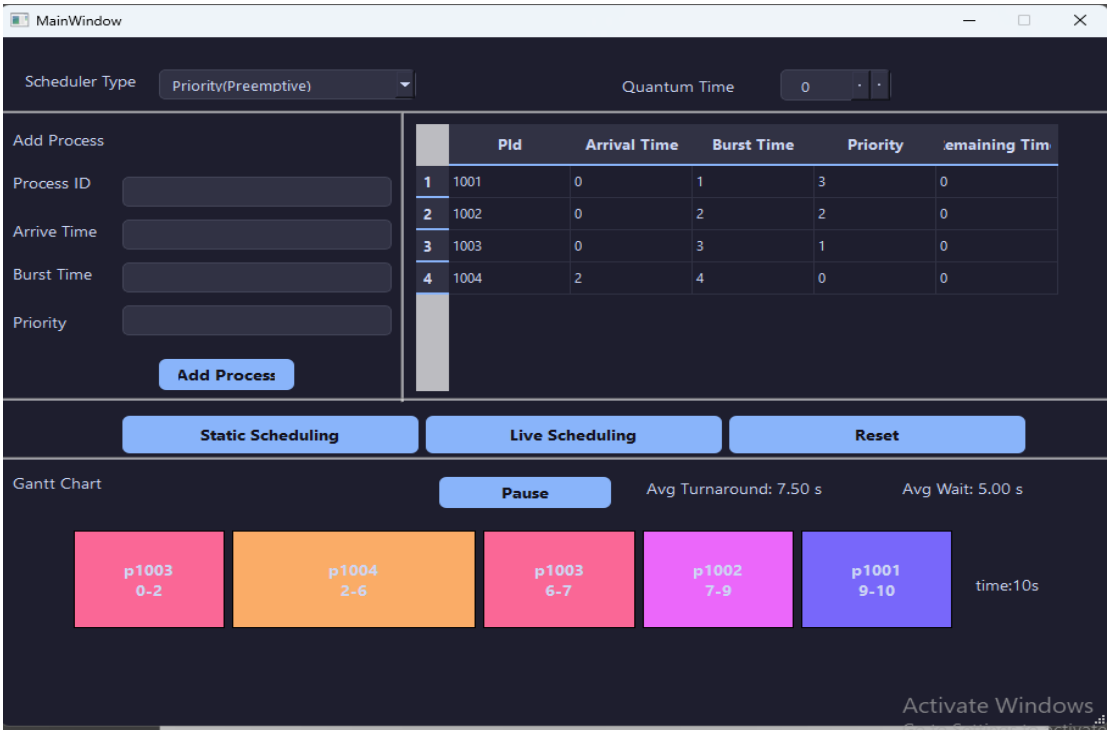




8) priority non preemptive



9) priority preemptive



Results & Discussion

The implemented CPU scheduling system was tested using different scheduling algorithms, and the results demonstrate clear differences in performance based on the selected method.

1. Comparison of Scheduling Algorithms

Each algorithm produced different waiting times and turnaround times depending on process characteristics:

- **FCFS (First Come First Serve):**
  This algorithm is simple but may lead to high average waiting time, especially when long processes arrive first.
- **SJF (Shortest Job First):**
  It achieved the lowest average waiting time in most test cases by prioritizing shorter processes. However, it may cause starvation for longer processes.
- **Priority Scheduling:**
  The execution order depended on process priority values. High-priority processes were executed first, but low-priority processes experienced delays.
- **Round Robin:**
  This algorithm provided fair CPU allocation among processes. Its performance depends heavily on the chosen time quantum. A small quantum increases context switching, while a large one makes it similar to FCFS.

2. Live vs Static Execution

The system supports both live and static scheduling modes:

- **Live mode** provides a real-time visualization of process execution, making it easier to understand how the CPU scheduler works step by step.
- **Static mode** executes all processes instantly and is useful for quick analysis and comparison.

3. Visualization and User Experience

The Gantt chart successfully visualizes process execution in a clear and dynamic way. Each process is assigned a unique color, making it easy to track execution order and duration. The remaining burst time table also updates in real time, improving system transparency.

4. Performance Metrics

The system calculates:

- Average Waiting Time
- Average Turnaround Time

These metrics allow users to evaluate the efficiency of each scheduling algorithm and compare their performance under different scenarios.

5. Overall Observation

The project successfully demonstrates how different CPU scheduling algorithms behave under the same set of processes. It provides a clear understanding of trade-offs between fairness, efficiency, and response time.

Conclusion

This project successfully implements a CPU scheduling simulator that includes multiple scheduling algorithms such as FCFS, SJF, Priority Scheduling, and Round Robin. The system provides both live and static simulation modes, allowing users to observe and analyze process execution in real time.

The graphical user interface built using Qt enhances usability by allowing easy input of processes and clear visualization of scheduling results through a dynamic Gantt chart. Additionally, the system calculates important performance metrics such as average waiting time and average turnaround time, which help in evaluating the efficiency of each algorithm.

Overall, this project provides a practical understanding of CPU scheduling concepts in operating systems and demonstrates the differences between various scheduling techniques in terms of performance and behavior.